

Flood Probability Prediction API

Capstone Project Proposal

July 2026

1. Certificate Name

Abdirahman Hussein Jama

2. Project Title and Description

Title: Flood Probability Prediction API

Floods are among the most destructive natural disasters, causing thousands of deaths and billions of dollars in economic losses each year. Early warning systems that can accurately predict flood risk are essential for disaster preparedness, evacuation planning, and infrastructure protection.

This project builds a machine learning model that predicts the probability of flooding in a region based on environmental, infrastructural, and socio-economic factors. The final deliverable is a REST API that accepts feature data as JSON and returns a flood probability score, useful for integration into early warning dashboards, emergency management systems, or urban planning tools.

3. Problem Type

Regression — predicting a continuous numeric value: FloodProbability.

The target column is FloodProbability, which ranges from approximately 0.285 to 0.725. This is supervised learning: the model trains on historical data where the flood probability is already known.

4. Dataset

Property	Details
Source	Kaggle — "Flood Prediction Dataset" by Naiya Khalid
Link	kaggle.com/datasets/naiyakhalid/flood-prediction-dataset
Size	50,000 rows × 21 columns
Target	FloodProbability — continuous probability of flooding (float)
Features	20 numeric variables covering weather, land, infrastructure, human, and environmental factors

Main Features

Feature	Description
MonsoonIntensity	Volume of rain during monsoon season
TopographyDrainage	Drainage capacity based on regional topography
RiverManagement	Quality and effectiveness of river management practices
Deforestation	Extent of deforestation in the area
Urbanization	Level of urbanization (impermeable surfaces increase runoff)
ClimateChange	Impact of climate change on the region
DamsQuality	Quality and maintenance status of dams
Siltation	Extent of siltation in rivers and reservoirs
DrainageSystems	Quality of drainage infrastructure
CoastalVulnerability	Susceptibility of coastal areas to flooding
PopulationScore	Population density in the region
Watersheds	Watershed characteristics
AgriculturalPractices	Types and sustainability of farming
Encroachments	Degree of construction on flood plains
IneffectiveDisasterPreparedness	Lack of emergency plans and warning systems
Landslides	Susceptibility to landslides
WetlandLoss	Extent of wetland destruction
InadequatePlanning	Quality of urban and regional planning
PoliticalFactors	Political influence on flood management
DeterioratingInfrastructure	Condition of existing infrastructure

Preprocessing Plan

All columns are numeric (int64 type) with no missing values. Preprocessing will include:

- Outlier detection and treatment using IQR (Interquartile Range) with a 1.5× threshold
- Feature scaling using MinMaxScaler or StandardScaler
- Train/test split: 80/20 stratified by target distribution

5. Algorithms I Plan to Train

#	Algorithm	Why It Fits
1	Linear Regression	Bootcamp baseline for regression — fast, interpretable, and provides a simple benchmark to compare against more complex models
2	Random Forest	Bootcamp ensemble method — handles non-linear relationships well, reduces overfitting versus a single decision tree, and gives feature-importance insights
3	Gradient Boosting	Researched — sequential ensemble that often achieves state-of-the-art performance on structured/tabular data by iteratively correcting prior errors

This meets the minimum of three distinct algorithms. Two (Linear Regression, Random Forest) come from the bootcamp. Gradient Boosting will be researched via scikit-learn documentation. XGBoost may be added as a fourth algorithm if time allows.

6. Evaluation Plan

Metrics for all models (on the same held-out test set):

- RMSE (Root Mean Squared Error) — penalizes large errors more heavily; interpretable in the same units as the target
- R^2 (Coefficient of Determination) — proportion of variance explained by the model; intuitive for comparing model fit
- MAE (Mean Absolute Error) — average absolute error; robust to outliers

Best-Model Rule

Rank all trained models by lowest RMSE on the test set. RMSE is appropriate here because over-predicting or under-predicting flood probability by a large margin both have serious consequences — false alarms cause unnecessary evacuations and economic disruption, while missed floods cost lives. If two models tie on RMSE, break the tie with higher R^2 .

A comparison table will be printed with one row per algorithm, and only the winning model will be saved and deployed.

7. Deployment Sketch

Component	Choice
Framework	FastAPI — modern, fast, auto-generates interactive API docs at /docs
Endpoint	POST /predict
Input	JSON with all 20 feature fields (numeric values)

Component	Choice
Output	JSON with flood_probability (float) and optional confidence interval

Input JSON Example

```
{
  "MonsoonIntensity": 7.5,
  "TopographyDrainage": 6.2,
  "RiverManagement": 5.8,
  "Deforestation": 4.1,
  "Urbanization": 8.3,
  "ClimateChange": 6.7,
  "DamsQuality": 5.5,
  "Siltation": 4.9,
  "DrainageSystems": 6.1,
  "CoastalVulnerability": 7.0,
  "PopulationScore": 5.4,
  "Watersheds": 6.8,
  "AgriculturalPractices": 4.3,
  "Encroachments": 5.2,
  "IneffectiveDisasterPreparedness": 3.9,
  "Landslides": 4.7,
  "WetlandLoss": 5.1,
  "InadequatePlanning": 4.8,
  "PoliticalFactors": 5.0,
  "DeterioratingInfrastructure": 4.4
}
```

Output JSON Example

```
{
  "flood_probability": 0.62,
  "risk_level": "High"
}
```

The API loads the best saved model from `models/best_model.pkl` plus the scaler artifact (`models/scaler.pkl`).

8. Repository Plan

```
flood-prediction-api/
├── dataset/
│   └── flood.csv           # Raw dataset (50k rows)
├── src/
│   ├── preprocess.py      # Cleaning, scaling, train/test split
│   └── train.py           # Train all models, compare, save best
└── api/
```

```
|   └─ app.py                # FastAPI app with /predict endpoint
└─ models/
    └─ best_model.pkl        # Saved best performing model
    └─ scaler.pkl            # Saved scaler for inference
    └─ model_comparison.csv   # Comparison table results
└─ notebooks/
    └─ eda.ipynb             # Exploratory data analysis
└─ tests/
    └─ test_api.py           # API unit tests
└─ README.md                 # Project overview and setup instructions
└─ requirements.txt          # Python dependencies
└─ project_paper.md          # 3-5 page project report
```

Planned Run Commands

```
python src/preprocess.py    # Prepare data
python src/train.py          # Train models, print comparison table, save
winner
uvicorn api.app:app --reload # Start API locally
```